UNIVERSIDAD DEL VALLE DE GUATEMALA

Data Science
Sección – 20
Ing. Mario Barrientos



Avances de proyecto 2

María José Girón Isidro - 23559
Daniela Ramírez de León - 23053
Leonardo Dufrey Mejía Mejía - 23648
Cindy Mishelle Gualim Perez - 21226
José Pablo Donado López - 22684

GUATEMALA, 8 de octubre de 2026

1. Introducción y descripción del problema

El reconocimiento automático del deletreo manual del lenguaje de señas estadounidense (American Sign Language, ASL) es un problema de modelado de secuencias, no de clasificación de imágenes: el significado depende de la forma de la mano y de cómo cambia en el tiempo. Se estima que entre el 12 % y el 35 % del discurso en ASL es deletreo manual (Padden & Gunsauls, 2003), usado para nombres propios, siglas y términos técnicos. Además, se ejecuta a unas 5 o 6 letras por segundo, con coarticulación entre letras y con letras de forma parecida (M, N y T).

En 2023, Google publicó en Kaggle el conjunto American Sign Language Fingerspelling Recognition, con más de tres millones de caracteres deletreados por más de cien firmantes sordos. Por privacidad y tamaño (158 GB), los datos no son video sino secuencias de landmarks de manos, rostro y pose extraídos con MediaPipe. Cada secuencia tiene una frase objetivo, pero no se indica en qué cuadro empieza o termina cada letra.

El análisis exploratorio del proyecto (Avance 1) aporta restricciones concretas para elegir modelos:

- Volumen y variabilidad: 67,208 secuencias de 94 participantes. La longitud de secuencia tiene mediana de 147 cuadros y cola hasta ~750; la frase va de 1 a 31 caracteres (mediana de 17).
- Faltantes: la mano derecha falta en ~54.5 % de los cuadros (rostro ~0.7 %). Se aplicó forward-fill y se descartaron secuencias con mano detectada en menos del 10 % de sus cuadros.
- Dimensión: se pasó de 1,630 a 226 columnas al eliminar los 468 puntos del rostro.
- Desbalance: unas pocas letras (e, a, o, r, n, t) concentran la mayoría de apariciones; j, q, z y varios símbolos casi no tienen muestras. Los participantes aportan entre 1 y 1,535 secuencias.

El objetivo de esta revisión bibliográfica es comparar cinco familias de modelos (CNN, LSTM, BiLSTM, GRU y Transformers): cómo funcionan, en qué problemas similares se han aplicado, qué ventajas y limitaciones tienen, y cuáles son los más adecuados, de forma preliminar, para transcribir deletreo manual en ASL a partir de landmarks..

2. Revisión bibliográfica de LSTM

Una Long Short-Term Memory (LSTM) está compuesta por redes neuronales recurrentes (RNN) y está diseñada para aprender dependencias de largo plazo y reducir el problema del desvanecimiento del gradiente presente en las redes recurrentes tradicionales (Hochreiter & Schmidhuber, 1997). Al procesar una secuencia utilizando esta arquitectura,a se puede encontrar una celda de memoria y tres compuertas de entrada, olvido y salida que regulan qué información conservar, actualizar o borrar.

En el caso del deletreo manual de ASL, una LSTM puede procesar los landmarks de las manos cuadro por cuadro para identificar los movimientos y transiciones entre letras.

Aplicaciones en problemas similares

- Buribayev et al. (2025) proponen un sistema de reconocimiento del lenguaje de señas kazajo que combina puntos clave extraídos mediante YOLO, información de flujo óptico y una red BiLSTM. Esta arquitectura permite analizar la información temporal en ambas direcciones y aprovechar los cambios de movimiento entre cuadros, demostrando la utilidad de las variantes de LSTM para el reconocimiento de gestos secuenciales.
- Graves y Schmidhuber (2005) evalúan redes LSTM bidireccionales para la clasificación de fonemas en secuencias de voz. Aunque no trabajan con lenguaje de señas, demuestran la aplicación de estas arquitecturas al reconocimiento de patrones temporales, un problema relacionado con la identificación de caracteres en secuencias continuas de deletreo manual.
- Shi et al. (2021) desarrollan un sistema para detectar segmentos de deletreo manual en videos continuos de ASL. Proponen un modelo de aprendizaje multitarea que integra detección, estimación de pose y reconocimiento de secuencias de letras. Su propuesta supera los métodos alternativos evaluados y demuestra la importancia de combinar información visual y temporal para identificar correctamente los segmentos de deletreo manual.
- Patil et al. (2024) implementan un sistema de conversión de lenguaje de señas a texto en tiempo real utilizando LSTM para reconocer gestos del alfabeto ASL. Entrenan modelos para distintos grupos de letras y reportan una exactitud del 97 % para el grupo P–Z y del 96 % para G–O. También identifican dificultades al distinguir letras con configuraciones similares, como G/H y M/N.

Ventajas

- Las LSTM pueden aprender dependencias temporales entre cuadros consecutivos y conservar información de movimientos anteriores, lo que permite modelar las transiciones entre configuraciones de las manos (Hochreiter & Schmidhuber, 1997).
- Sus mecanismos de memoria y compuertas reducen las dificultades asociadas al desvanecimiento del gradiente, facilitando el aprendizaje de patrones en secuencias extensas.
- Pueden procesar directamente secuencias de landmarks y combinarse con CTC para generar transcripciones de longitud variable sin necesitar una correspondencia explícita entre cada cuadro y cada carácter (Graves et al., 2006).
- Las LSTM permiten aprender patrones temporales y conservar información de movimientos anteriores. Patil et al. (2024) aplican esta capacidad al reconocimiento de gestos del alfabeto ASL, obteniendo exactitudes superiores al 90 % en la mayoría de los grupos de letras evaluados.

Limitaciones

- El procesamiento recurrente depende de los estados anteriores, lo que dificulta la paralelización temporal y puede aumentar el tiempo de entrenamiento en comparación con arquitecturas convolucionales (Bai et al., 2018).
- El reconocimiento puede verse afectado por la similitud entre configuraciones de las manos. Patil et al. (2024) reportan confusiones entre letras como G/H, I/J y M/N, además de diferencias en el tiempo necesario para reconocer determinados gestos.
- La variabilidad de los participantes, los movimientos y las condiciones de grabación dificulta el reconocimiento en entornos reales. Shi et al. (2021) evidencian la necesidad de detectar correctamente los segmentos de deletreo manual antes de transcribirlos, mientras que Patil et al. (2024) destacan los desafíos asociados con distintos estilos de ejecución.

Relevancia para el proyecto: Esta propuesta es una alternativa adecuada para el reconocimiento automático del deletreo manual de ASL, debido a su capacidad para modelar movimientos y transiciones entre letras. Los resultados de Patil et al. (2024) respaldan su aplicación al reconocimiento de gestos del alfabeto, mientras que Shi et al. (2021) destacan la importancia de considerar la información temporal y la detección de segmentos en secuencias continuas.

3. Revisión bibliográfica de CNN

Una CNN aplica filtros pequeños con pesos compartidos que detectan patrones locales (LeCun et al., 1998; introducción en Zhang et al., 2023, sec. 7.1). Sobre imágenes, el filtro recorre el espacio (CNN 2D). Sobre landmarks, el filtro recorre el eje del tiempo (CNN 1D): las 226 columnas por cuadro funcionan como canales y cada filtro aprende patrones de unos pocos cuadros, como la velocidad con que baja un dedo. Las redes convolucionales temporales (TCN) añaden convoluciones causales y dilatadas para ampliar el campo receptivo (Bai et al., 2018).

Aplicaciones en problemas similares

- Bai et al. (2018) comparan una TCN genérica con LSTM en varias tareas de secuencias y encuentran que la convolucional supera a las recurrentes y tiene memoria efectiva más larga.
- Shi et al. (2019) usan una CNN tipo AlexNet sobre los cuadros de video para extraer rasgos de apariencia que luego procesa una LSTM con CTC.
- Korotaev et al. (2025), con HandReader, emplean un ResNet-34 con módulo de desplazamiento temporal para video y un codificador temporal para landmarks de MediaPipe, con decodificador BiGRU y CTC. Con landmarks obtienen 69.3 % y 72.4 % de exactitud por letra en ChicagoFSWild y ChicagoFSWild+; combinando video y landmarks suben a 72.9 % y 75.6 %.

- Kamble (2025) antepone una CNN a la LSTM para extraer rasgos espaciales de los 543 landmarks de cada cuadro.

Ventajas

- Se paraleliza en el tiempo, por lo que entrena e infiere más rápido que una red recurrente.
- Capta bien los micro-movimientos locales y es estable al entrenar.
- Con *stride* reduce la longitud de la secuencia antes de un modelo recurrente o de atención, lo que abarata el cómputo en secuencias de hasta ~750 cuadros.

Limitaciones

- Cada capa solo ve una ventana corta; para relacionar cuadros lejanos exige profundidad o dilataciones.
- Por sí sola no produce una frase de longitud variable: necesita una pérdida como CTC o un decodificador encima.
- Las CNN 2D sobre píxeles requieren video, que la competencia no distribuye; con landmarks solo aplica la versión 1D.

Relevancia para el proyecto: una CNN 1D es útil como extractor inicial y para reducir la longitud temporal, más que como modelo único.

4. Revisión bibliográfica de BiLSTM (Cindy)

Una BiLSTM (Bidirectional Long Short-Term Memory) combina dos LSTM que recorren la misma secuencia en sentidos opuestos: una lee los cuadros de izquierda a derecha y la otra de derecha a izquierda. Las salidas de ambas se concatenan en cada posición, de modo que cada cuadro queda representado con información del pasado y del futuro (Schuster & Paliwal, 1997; Graves y Schmidhuber, 2005).

En el deletreo manual de ASL, esta estructura permite que la representación de una letra dependa también de las configuraciones que aparecen después en la secuencia. Esto ayuda cuando la transición entre dos letras de forma parecida, como M y N, se distingue mejor observando el cuadro siguiente.

Aplicaciones en problemas similares

- Buribayev et al. (2025) usan una BiLSTM sobre puntos clave extraídos con YOLO e información de flujo óptico para reconocer lengua de señas kazaja. El modelo aprovecha los cambios de movimiento entre cuadros en ambos sentidos.
- Graves y Schmidhuber (2005) comparan LSTM bidireccionales con otras arquitecturas en la clasificación de fonemas. Aunque el dominio es la voz, el resultado muestra que el contexto de ambos lados mejora la clasificación de patrones temporales frente a los modelos unidireccionales.

- Schuster y Paliwal (1997) formalizan las redes recurrentes bidireccionales, que son la base de la variante LSTM utilizada en los trabajos anteriores.

Ventajas

- Captura contexto en ambas direcciones, lo que ayuda a manejar la coarticulación entre letras (Schuster y Paliwal, 1997).
- Conserva el mecanismo de compuertas de la LSTM, que reduce el desvanecimiento del gradiente en secuencias largas (Hochreiter & Schmidhuber, 1997).
- Puede combinarse con CTC para generar transcripciones de longitud variable sin alinear cada cuadro con una letra (Graves et al., 2006).

Limitaciones

- Necesita la secuencia completa antes de procesarla, por lo que no es adecuada para reconocimiento en tiempo real sin adaptaciones.
- Su costo es mayor que el de una LSTM unidireccional; la tabla comparativa del apartado 7 lo estima en unas dos veces el de una LSTM.
- Al seguir siendo recurrente, depende de los estados anteriores y de los posteriores, lo que limita la paralelización del entrenamiento frente a las CNN y los Transformers (Bai et al., 2018).
- Las cifras reportadas en la literatura provienen de vocabularios pequeños y datos propios, por lo que no son directamente comparables con nuestra tarea de frases completas.

Relevancia para el proyecto: La BiLSTM se incluye como variante con CTC para medir si las compuertas adicionales compensan su costo frente a la BiGRU que sostiene el modelo principal (apartado 8). Su necesidad de la secuencia completa la convierte en una comparación, no en el modelo final.

5. Revisión bibliográfica de Transformers (Daniela)

El Transformer (Vaswani et al., 2017) procesa una secuencia completa mediante autoatención. Cada posición calcula su relación con todas las demás y forma una representación ponderada del contexto. La atención multi-cabeza repite este cálculo en varios subespacios en paralelo, de modo que cada cabeza puede especializarse en distintos patrones. La codificación posicional añade el orden temporal, que la atención por sí sola no conoce. La arquitectura original combina un codificador, que construye representaciones de la entrada, con un decodificador que genera la salida token por token. En el deletreo manual de ASL, cada cuadro de landmarks se proyecta linealmente y actúa como un token. Así, el modelo puede relacionar cualquier cuadro con cualquier otro y captar el contexto global de la frase.

5.1 Aplicaciones en problemas similares

- La solución ganadora de la competencia de Kaggle usó un codificador Squeezeformer (Kim et al., 2022), adaptado para recibir landmarks de MediaPipe, y un decodificador Transformer de dos capas (Henkel, 2023). Se apoyó en aumentaciones como CutMix, FingerDropout y TimeStretch. Según su autor, la aumentación fue clave porque los firmantes tienen distintos estilos y velocidades de deletreo (NVIDIA, 2023).
- Georg et al. (2025), con FSboard, alimentan el codificador de ByT5-Small con 85 landmarks de MediaPipe y obtienen 11.1 % de error por carácter (CER) en firmantes no vistos. Sin preentrenamiento, el CER sube a 33.8 %. Al quitar el rostro solo empeora a 12.0 %.
- Fayyazsanavi et al. (2023) entrenan un Transformer codificador-decodificador sobre los 21 landmarks de la mano, combinando CTC y entropía cruzada. Alcanzan 66.3 % y 71.1 % de exactitud por letra en ChicagoFSWild y ChicagoFSWild+.
- Camgoz et al. (2020) entrenan un Transformer de forma conjunta para reconocimiento, con CTC, y para traducción de lengua de señas continua. Obtienen 24.49 % de WER en PHOENIX14T sin necesitar alineación temporal.

5.2 Ventajas

- Capta contexto global en ambas direcciones, lo que ayuda con la coarticulación entre letras.
- Se paraleliza en el tiempo, por lo que entrena más rápido en GPU que una red recurrente (Vaswani et al., 2017).
- Es la familia con mejores resultados documentados en este conjunto de datos y en FSboard (Georg et al., 2025; Henkel, 2023).
- Es compatible con CTC (Graves et al., 2006) y con decodificadores autorregresivos, y ambos se pueden combinar (Camgoz et al., 2020).

5.3 Limitaciones

- El costo de la autoatención crece de forma cuadrática con la longitud de la secuencia (Vaswani et al., 2017). Una secuencia de 750 cuadros implica unas 26 veces más pares de atención que una de 147. Por eso Squeezeformer reduce la resolución temporal en sus capas intermedias (Kim et al., 2022).
- Requiere muchos datos o preentrenamiento. En FSboard, entrenar desde cero triplicó el CER (Georg et al., 2025).
- Es propenso al sobreajuste con pocos firmantes. En FSboard, un modelo más grande empeoró el CER a 13.3 % (Georg et al., 2025). Además, la atención bidireccional necesita la secuencia completa, así que no sirve para tiempo real sin adaptaciones.

5.4 Relevancia para el proyecto

El Transformer se ajusta al problema porque no hay alineación cuadro-letra, lo que se resuelve con CTC, y porque las frases dependen del contexto. Los faltantes de la mano derecha (~54.5 %) pueden manejarse con máscaras de atención y aumentaciones como FingerDropout. Los resultados de FSboard también respaldan haber eliminado los puntos del rostro. Por eso se

mantiene como el modelo de mayor techo, en una versión reducida: un codificador pequeño con CTC, recorte o submuestreo de las secuencias largas y validación separada por participante.

6. **Revisión bibliográfica de GRU (José)**

Según Zhang y Jiang (2024), con la llegada de modelos de Redes Neuronales Recurrentes (RNN), el uso de imágenes RGB para analizar videos de deletreo manual dejó en segundo plano las tecnologías que requerían de un hardware específico para comunicarse con el ASL dado y capturar la semántica del movimiento (e.g., sensores, cámaras de infrarrojo).

Uno de los mayores retos en la traducción del lenguaje de señas es el problema la epéntesis del movimiento: cuando la mano y la muñeca se trasladan de una a otra palabra, lo que forma parte de una buena porción del lapso del video. En este caso, se tiene que reconocer el milisegundo en el que una palabra se ha terminado y seguir cuando la siguiente palabra se gesticula (retención apical). A lo anterior se aúna la velocidad y variaciones de las manos entre personas (Becerril-Carrillo et al., 2026)

Dentro de estos modelos de aprendizaje automático, se encuentra la Unidad Recurrente con Compuertas GRU, que compensa la propagación de error hacia atrás y la disminución de gradiente del modelo tradicional de RNN: LSTM. El modelo de GRY evita el aprendizaje de dependencias a largo plazo y para ello cuenta con dos compuertas:

1. Compuerta de actualización (Update Gate): regula dinámicamente la retención de la memoria a largo plazo. Define qué porción del estado oculto anterior se preserva intacta y qué fracción se reescribe con la nueva activación candidata. Esta compuerta reemplaza al conjunto de las compuertas de olvido y de entrada del LSTM.
2. Compuerta de restablecimiento (Reset Gate): Modula retrospectivamente cuánto de la memoria pasada es relevante para el cálculo del estado candidato actual. Si la compuerta de restablecimiento se aproxima a cero, la unidad GRU descarta lo visto y lee el fotograma actual como si fuera el inicio de una nueva secuencia léxica, que es ideal para delimitar el final de una palabra deletreada e ignorar los “silencios” visuales.

Este modelo es mejor que el algoritmo de LSTM en distintos aspectos, empezando porque tiene menos densidad paramétrica, lo que implica menos costos computacionales; un menor riesgo de sobreajuste sobre conjuntos de datos pequeños y viabilidad para implementar en entrenamientos de datos masivos gracias a que los modelos de GRU convergen en la mitad del tiempo (Myagila, Nyambo y Dida, 2025).

A continuación, se presenta un cuadro comparativo de las diferencias principales entre la arquitectura de LSTM y GRU para modelos series de tiempo con base en el análisis de Zhang y Jiang (2024):

Característica arquitectónica	**LSTM (Long Short-Term Memory)**	**GRU (Gated Recurrent Unit)**	**Impacto en el ASLR**

Complejidad de compuertas	3 compuertas (Olvido, entrada, salida)	2 compuertas (Actualización, restablecimiento)	GRU minimiza el tamaño del modelo final para dispositivos limitados (edge devices).
Separación de estado	Estado oculto (hidden) y Estado de celda (Cell) separados	Ambos estados están fusionados en un solo vector	Reducción de la huella de memoria RAM durante la inferencia secuencial en GRU.
Velocidad de convergencia	Menor, requiere más épocas (epochs) y cálculo tensorial	Mayor, típicamente iterando el doble de rápido por época	GRU permite iteraciones investigativas aceleradas sobre grandes conjuntos de datos de video.
Preferencia topológica	Favorecida para secuencias extremadamente largas o complejas	Demuestra superioridad en secuencias de complejidad baja a moderada y datasets de bajo volumen	GRU es altamente sinérgico con dactilología, donde las ventanas transicionales epentéticas son localizadas.

En general, las arquitecturas recurrentes de compuertas modulares se consideran componentes troncales de la dactilología. En lo que Albanie et al. (2020), consideran un teatro biofísico de alta entropía visual temporal, los abordajes convolucionales atómicos cuadro por cuadro son obsoletos, abriendo paso al imperativo del modelamiento con estado transitorio dependiente del tiempo.
La Unidad Recurrente con Compuertas (GRU) proporciona a las redes visuales un equilibrio matemático supremo frente al pesado algoritmo Long Short-Term Memory (LSTM). Al sintetizar el tránsito semántico temporal de tres puertas moduladoras hacia únicamente dos operaciones de enrutamiento y de olvido matricial unificadas, el modelo GRU no solamente requiere menos cómputo y convergencia de gradiente hiper-acelerada a lo largo del entrenamiento por retropropagación (Zhang y Jiang, 2024), sino que su matriz funciona como un regulador pasivo que previene sobreajustes perjudiciales sobre bases de entrenamiento modestas, tal como las de deletreo manual.

En vez de pruebas con sujetos aislados cautivos (de un mismo color de piel incluso) en estudios de iluminación controlada con reportes ideales de certeza del 99%, con los modelos de GRU se obtiene realidades operativas concretas de 65% de Precisión de Letra (LAcc), empujando fronteras con la inserción superpuesta de atención algorítmica recursiva (Attentional Visual Iteration) y anclaje lingüístico (Zhang y Jiang, 2024).

Finalmente, la extrema eficiencia de los coeficientes tensoriales del modelo ha fomentado su migración directa desde repositorios en clústeres estáticos hacia arquitecturas para ciegos y sordos (como el anillo kinético SpellRing), alcanzando cotas de Tasa de Error de Palabra (WER) fenomenalmente minúsculas de 0.099 procesando el léxico interactivo instantáneamente en el borde. Este hardware aunado a la arquitectura con un filtrado de articulaciones temporal proporcionado por GRU/BiGRU, y el decodificador latente probabilístico CTC sentan las bases para realizar una traducción en sincronía con la rapidez del deletreo manual (Hyunchul et al., 2025).

7. Tabla comparativa de los cinco modelos (Mejía y Cindy)

Criterio	**CNN (1D/TCN)**	**LSTM**	**BiLSTM**	**GRU**	**Transforme r**
Cómo capta el tiempo	Filtros locales sobre ventanas	Estado recurrente con celda	Dos LSTM, una por sentido	Estado recurrente con 2	Autoatención sobre toda la secuencia

				compuertas	
Contexto	Local; crece con profundidad y dilatación	Solo pasado	Pasado y futuro	Solo pasado (BiGRU: ambos)	Global
Paralelo en el tiempo	Sí	No	No	No	Sí
Costo relativo	Bajo	Medio	Alto (≈2× LSTM)	Medio-bajo	Alto en secuencias largas
Datos que necesita	Pocos a moderados	Moderados	Moderados	Pocos a moderados	Muchos, con aumentación
Longitud variable de salida	Requiere CTC o decodificador	Con CTC	Con CTC	Con CTC	Con CTC o decodificador
Streaming (tiempo real)	Sí (convolución causal)	Sí	No	Sí	No sin adaptaciones
Evidencia en deletreo o landmarks	HandReader: 69.3 % letras con landmarks (ChicagoFSWild)	SLRNet: 86.7 % en 26 letras ASL (datos propios)	Kazakh SL: 47 clases, mejora inconsistente	MOPGRU: ~95 % en 13 signos (datos propios)	Ganador de esta competencia; FSboard: 11.1 % CER
Limitación principal	Contexto corto por sí sola	Secuencial y unidireccional	Necesita secuencia completa	No supera de forma consistente a LSTM	Intensivo en datos y cómputo

8. Selección preliminar de modelos y justificación

Rol	Modelo	Justificación con los hallazgos del EDA
Principal	CNN 1D + BiGRU + CTC	La mediana es de 147 cuadros con cola hasta ~750 y la frase tiene 1 a 31 caracteres: la CNN con *stride* acorta la secuencia y CTC evita alinear letras. La BiGRU usa contexto de ambos lados con menos parámetros que una BiLSTM. Es la estructura de HandReader (Korotaev et al., 2025).
Mayor techo	Transformer (codificador pequeño + CTC)	La solución ganadora de la competencia es de esta familia (Henkel, 2023). Se prueba en versión reducida para respetar el cómputo disponible.
Línea base	LSTM unidireccional + CTC	Es la referencia más citada en reconocimiento de señas con landmarks (Kamble, 2025; Srivastava et al., 2024) y permite medir cuánto aporta el contexto futuro.
Variante	BiLSTM + CTC	Comparación directa con BiGRU para ver si las compuertas extra compensan el costo.
Solo ablación	GRU unidireccional	Sin contexto futuro, ya queda cubierta por la línea base LSTM y por la BiGRU.

9. Conclusiones y revisión de referencias APA

La literatura apunta a que, para transcribir deletreo manual desde landmarks, lo que más pesa es combinar un extractor local, un modelo con contexto de ambos lados y una pérdida sin alineación, y no la elección de una sola familia.

- Las CNN, LSTM, BiLSTM y GRU se han probado sobre todo en vocabularios pequeños y datos propios, así que sus cifras no se pueden comparar entre sí ni con nuestra tarea de frases completas.
- La evidencia más cercana a nuestro problema viene de HandReader (CNN + BiGRU + CTC) y de la solución ganadora de la competencia (codificador Squeezeformer + decodificador Transformer).

- CTC (Graves et al., 2006; explicación visual en Hannun, 2017) resuelve que la etiqueta sea la frase y no una letra por cuadro; es el punto en común de casi todas las arquitecturas revisadas.
- Los datos que ya exploramos, que son secuencias de hasta casi 750 cuadros, con 54.5% de ellos faltantes en la mano derecha y desbalance de participantes o caracteres, pesan más en el diseño del entrenamiento que la elección entre LSTM y GRU.
- Siguiente paso: entrenar la CNN 1D + BiGRU y la LSTM sobre data/processed/ con CTC y comparar el error por carácter en un conjunto de validación separado por participante.

Referencias

- Albanie, S., Varol, G., Momeni, L., Afouras, T., Chung, J.S., Fox, N., Zisserman, A.: Bsl-1k: Scaling up co-articulated sign language recognition using mouthing cues. In: European conference on computer vision. pp. 35–53. Springer (2020)
- Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv. https://arxiv.org/abs/1803.01271
- Becerril-Carrillo, A. d. J., Selley-Rojas, H. J., & Guevara-Martínez, E. (2026). Recurrent Neural Networks for Mexican Sign Language Interpretation in Healthcare Services. Sensors, 26(1), 27. https://doi.org/10.3390/s26010027
- Buribayev, Z., Aouani, M., Zhangabay, Z., Yerkos, A., Abdirazak, Z., & Zhassuzak, M. (2025). Enhancing Kazakh sign language recognition with BiLSTM using YOLO keypoints and optical flow. Applied Sciences, 15(10), Article 5685. https://doi.org/10.3390/app15105685
- Camgoz, N. C., Koller, O., Hadfield, S., & Bowden, R. (2020). Sign language transformers: Joint end-to-end sign language recognition and translation. arXiv. https://arxiv.org/abs/2003.13830
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (pp. 1724–1734). Association for Computational Linguistics. https://aclanthology.org/D14-1179
- Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv. https://arxiv.org/abs/1412.3555
- Fayyazsanavi, P., Nejatishahidin, N., & Košecká, J. (2023). Fingerspelling PoseNet: Enhancing fingerspelling translation with pose-based transformer models [Preprint]. arXiv. https://arxiv.org/abs/2311.12128
- Georg, M., Tanzer, G., Uboweja, E., Hassan, S., Shengelia, M., Sepah, S., Forbes, S., & Starner, T. (2025). FSboard: Over 3 million characters of ASL fingerspelling collected via smartphones. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 13897–13906). https://openaccess.thecvf.com/content/CVPR2025/html/Georg_FSboard_Over_3_Million_Characters_of_ASL_Fingerspelling_Collected_via_CVPR_2025_paper.html
- Google. (2023). American Sign Language fingerspelling recognition [Competencia]. Kaggle. https://www.kaggle.com/competitions/asl-fingerspelling
- Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J. (2006). Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In Proceedings of the 23rd International Conference on Machine Learning (pp. 369–376). https://www.cs.toronto.edu/~graves/icml_2006.pdf

- Graves, A., & Schmidhuber, J. (2005). Framewise phoneme classification with bidirectional LSTM and other neural network architectures. Neural Networks, 18(5–6), 602–610. https://archivedsite.air.in.tum.de/Main/Publications/Graves2005a.pdf
- Hannun, A. (2017). Sequence modeling with CTC. Distill, 2(11). https://doi.org/10.23915/distill.00008
- Henkel, C. (2023). 1st place solution to the Google – American Sign Language Fingerspelling Recognition competition [Repositorio de código]. GitHub. https://github.com/ChristofHenkel/kaggle-asl-fingerspelling-1st-place-solution
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735
- Hyunchul Lim, Nam Anh Dang, Dylan Lee, Tianhong Catherine Yu, Jane Lu, Franklin Mingzhe Li, Yiqi Jin, Yan Ma, Xiaojun Bi, François Guimbretière, and Cheng Zhang. (2025). SpellRing: Recognizing Continuous Fingerspelling in American Sign Language using a Ring. In Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25). Association for Computing Machinery, New York, NY, USA, Article 67, 1–17. https://doi.org/10.1145/3706598.3713721
- Kamble, S. (2025). SLRNet: A real-time LSTM-based sign language recognition system. arXiv. https://arxiv.org/abs/2506.11154
- Kim, S., Gholami, A., Shaw, A., Lee, N., Mangalam, K., Malik, J., Mahoney, M. W., & Keutzer, K. (2022). Squeezeformer: An efficient transformer for automatic speech recognition. Advances in Neural Information Processing Systems, 35, 9361–9373. https://arxiv.org/abs/2206.00888
- Korotaev, P., Surovtsev, P., Kapitanov, A., Kvanchiani, K., & Nagaev, A. (2025). HandReader: Advanced techniques for efficient fingerspelling recognition. arXiv. https://arxiv.org/abs/2505.10267
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278–2324. https://doi.org/10.1109/5.726791
- Myagila K, Nyambo DG and Dida MA (2025) Efficient spatio-temporal modeling for sign language recognition using CNN and RNN architectures. Front. Artif. Intell. 8:1630743. doi: 10.3389/frai.2025.1630743
- NVIDIA. (2023, 23 de octubre). Reading the signs: NVIDIAN wins Kaggle AI contest for sign language recognition [Entrada de blog]. NVIDIA Blog. https://blogs.nvidia.com/blog/2023/10/23/nvidia-life-kaggle-ai-contest-sign-language-recognition/
- Olah, C. (2015, 27 de agosto). Understanding LSTM networks [Entrada de blog]. colah's blog. https://colah.github.io/posts/2015-08-Understanding-LSTMs/
- Padden, C. A., & Gunsauls, D. C. (2003). How the alphabet came to be used in a sign language. Sign Language Studies, 4(1), 10–33.
- Patil, D., Patil, A., Patil, A., More, A., & Patil, S. S. (2024). Sign Language to Text Conversion. International Journal for Research in Applied Science and Engineering Technology. https://doi.org/10.22214/ijraset.2024.62147

- Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. IEEE Transactions on Signal Processing, 45(11), 2673–2681. https://doi.org/10.1109/78.650093
- Shi, B., Brentari, D., Shakhnarovich, G., & Livescu, K. (2021). Fingerspelling Detection in American Sign Language. 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 4164–4173. https://doi.org/10.1109/cvpr46437.2021.00415
- Shi, B., Del Rio, A. M., Keane, J., Brentari, D., Shakhnarovich, G., & Livescu, K. (2019). Fingerspelling recognition in the wild with iterative visual attention. In Proceedings of the IEEE/CVF International Conference on Computer Vision. https://arxiv.org/abs/1908.10546
- Srivastava, S., Singh, S., Pooja, & Prakash, S. (2024). Continuous sign language recognition system using deep learning with MediaPipe Holistic. arXiv. https://arxiv.org/abs/2411.04517
- Subramanian, B., Olimov, B., Naik, S. M., Kim, S., Park, K.-H., & Kim, J. (2022). An integrated mediapipe-optimized GRU model for Indian sign language recognition. Scientific Reports, 12, Article 11964. https://doi.org/10.1038/s41598-022-15998-7
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30. https://arxiv.org/abs/1706.03762
- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Dive into deep learning. Cambridge University Press. https://d2l.ai
- Zhang, Y., & Jiang, X. (2024). Recent Advances on Deep Learning for Sign Language Recognition. Computer Modeling in Engineering & Sciences, 139(3), 2399–2450. https://doi.org/10.32604/cmes.2023.045731